UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO
(UFRPE)
DEPARTAMENTO DE COMPUTAÇÃO
TÓPICOS AVANÇADOS EM INTELIGÊNCIA ARTIFICIAL (2026.2)

JOSÉ ALBÉRICO ROCHA DA SILVA

IMPLEMENTAÇÃO DO ZERO DE REDE NEURAL MULTICAMADAS (MLP) EM GRAFO DIRECIONADO: ARQUITETURA, DINÂMICA DE RETROPROPAGAÇÃO E OTIMIZAÇÃO DE HIPERPARÂMETROS

Artefato [5] · `mlp git repo` (`mlp·c`)

Relatório técnico apresentado ao Prof. Dr. Lucas Silva na disciplina de Tópicos Avançados em Inteligência Artificial da UFRPE, contemplando os 5 pontos de verificação do artefato `mlp·c`.

Recife – PE
2026

Sumário

1 Introdução e Fundamentação Conceitual **2**

1.1 Domínio Clínico e Particionamento Explícito 80/20 2

2 Desenho da Arquitetura e Dinâmica Matemática **2**

2.1 Representação Explícita em Grafo Direcionado (`nx.DiGraph`) 2

2.2 Propagação Direta (*Forward Pass*) 3

2.3 Retropropagação (*Backpropagation*) e Correção Analítica do Gradiente BCE 4

3 Otimização de Hiperparâmetros, Visualização de Pesos e Resultados **4**

3.1 Estudo de Ablação e Otimização de Acurácia e Perda 4

3.2 Justificativa dos Parâmetros Modificados 5

3.3 Visualização Numérica e Topológica dos Pesos Sinápticos 6

1 Introdução e Fundamentação Conceitual

O desenvolvimento de Redes Neurais Artificiais Multicamadas (*Multi-Layer Perceptrons* – MLPs) consolidou-se a partir da formulação do algoritmo de retropropagação de erros (*backpropagation*) por Rumelhart, Hinton e Williams [5] e da sistematização teórica do aprendizado profundo por Goodfellow, Bengio e Courville [2]. Na prática contemporânea, todavia, o uso exclusivo de bibliotecas de alto nível com diferenciação automática oculta os detalhes algébricos e topológicos que governam o fluxo de ativações e gradientes.

O presente trabalho documenta a implementação **do zero (*from scratch*)** do artefato `[5] mlp git repo` (`mlp·c`), na qual uma rede neural multicamadas completa é modelada explicitamente sobre um **grafo direcionado ponderado** (`networkx.DiGraph`) com operações numéricas em `NumPy` e interface interativa em tempo real construída em `Streamlit`.

1.1 Domínio Clínico e Particionamento Explícito 80/20

O problema abordado consiste na predição diagnóstica de Doença Arterial Coronariana a partir da base *Heart Disease* [1], composta por $N = 1.025$ registros clínicos e $d = 13$ atributos fisiológicos e laboratoriais (`age`, `sex`, `cp`, `trestbps`, `chol`, `fbs`, `restecg`, `thalach`, `exang`, `oldpeak`, `slope`, `ca` e `thal`), além do alvo binário `target` (0 = saudável, 1 = presença de cardiopatia).

Para assegurar rigor metodológico e ausência total de vazamento de dados (*zero data leakage*), o módulo `code/mlp/core_mlp/dataset.py` executa:

1. **Particionamento Explícito Estratificado (80% Treino / 20% Teste):** divisão determinística (`seed=42`) em 820 amostras de treinamento (80%, exportadas em `train_80_standardized.csv`) e 205 amostras de teste (20%, exportadas em `test_20_standardized.csv`), preservando a proporção de classes (51,3% positivos).

2. **Padronização Pós-Particionamento:** o estimador estatístico (`StandardScaler` $z = (x - \mu_{\text{treino}})/\sigma_{\text{treino}}$ ou `MinMaxScaler`) é ajustado (`fit_transform`) exclusivamente sobre as 820 amostras de treino e aplicado (`transform`) sobre as 205 amostras de teste.

2 Desenho da Arquitetura e Dinâmica Matemática

2.1 Representação Explícita em Grafo Direcionado (`nx.DiGraph`)

Diferentemente de implementações matriciais opacas, a classe `MLPGraph` (`code/mlp/core_mlp/graph`
instancia cada neurônio e cada sinapse como entidades explícitas de um grafo direcionado

$G = (V, E)$, conforme ilustrado no desenho arquitetural da Figura 1.

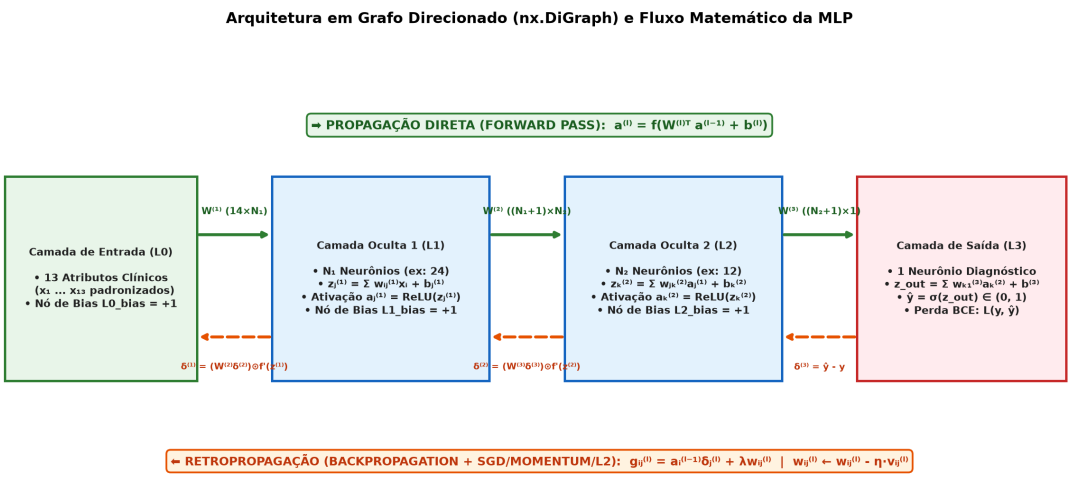


Figura 1: Desenho da arquitetura da MLP em grafo direcionado (`nx.DiGraph`) e fluxo matemático bidirecional (*Forward Pass* e *Backpropagation*).

O funcionamento estrutural do grafo organiza-se em três componentes:

- **Vértices Neurais** (`L{l}_N{j}`)**:** cada neurônio j da camada $l \in \{0, \ldots, L-1\}$ armazena os atributos `z` (potencial pré-ativação $z_j^{(l)}$), `value` (ativação $a_j^{(l)}$) e `delta` (erro retropropagado $\delta_j^{(l)}$).
- **Vértices de Bias** (`L{l}_bias`)**:** cada camada $l \in \{0, \ldots, L-2\}$ possui um nó de bias explícito com ativação constante $a_{\text{bias}}^{(l)} = +1$, conectado a todos os neurônios da camada $l+1$. Essa modelagem corrige uma limitação da versão preliminar, na qual a primeira camada oculta carecia de termo de bias.
- **Arestas Sinápticas** (`(u, v)`)**:** cada aresta direcionada armazena o peso sináptico atual `weight` (w_{uv}), o peso no início da época `prev_weight` ($w_{uv}^{(t-1)}$), a variação líquida na época `delta_weight` (Δw_{uv}), o gradiente instantâneo `gradient` ($\partial \mathcal{L}/\partial w_{uv}$) e a velocidade de momento `velocity` (v_{uv}).

2.2 Propagação Direta (*Forward Pass*)

Para cada amostra $\mathbf{x} \in \mathbb{R}^{13}$, os nós de entrada recebem $a_i^{(0)} = x_i$. Em seguida, para cada camada $l = 1, \ldots, L-1$, calcula-se o potencial pós-sináptico e a ativação não linear:

$$z_j^{(l)} = \sum_{i=1}^{N_{l-1}} w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \qquad a_j^{(l)} = f^{(l)}\left(z_j^{(l)}\right) \tag{1}$$

Nas camadas ocultas, o sistema suporta sete funções de ativação configuráveis em tempo real (`ReLU`, `Leaky ReLU`, `Tanh`, `Sigmoide`, `ELU`, `SELU` e `ReLU6`). Na camada de saída ($L-1$), emprega-se a função sigmoide $\hat{y} = \sigma(z_{\text{out}}) = (1 + e^{-z_{\text{out}}})^{-1}$.

2.3 Retropropagação (*Backpropagation*) e Correção Analítica do Gradiente BCE

A função objetivo adotada é a Entropia Cruzada Binária (*Binary Cross-Entropy* – BCE) com regularização L_2:

$$\mathcal{L}(\hat{y}, y) = -\Big[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})\Big] + \frac{\lambda}{2} \sum_{l,i,j} \left(w_{ij}^{(l)}\right)^2 \tag{2}$$

Na revisão da versão preliminar, constatou-se que o sinal de erro na saída estava implementado como $(\hat{y} - y)\sigma'(z_{\text{out}})$, expressão correspondente à perda quadrática (MSE) que introduz saturação prematura quando $\hat{y} \to 0$ ou $\hat{y} \to 1$. Pela regra da cadeia aplicada à BCE, o termo $\hat{y}(1 - \hat{y})$ cancela-se exatamente:

$$\delta^{(L-1)} = \frac{\partial \mathcal{L}}{\partial z_{\text{out}}} = \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}\right) \hat{y}(1 - \hat{y}) = \hat{y} - y \tag{3}$$

Para as camadas ocultas $l = L - 2, \ldots, 1$, o erro é retropropagado pelas arestas sucessoras de cada nó:

$$\delta_j^{(l)} = \left(\sum_{k=1}^{N_{l+1}} w_{jk}^{(l+1)} \delta_k^{(l+1)}\right) f'^{(l)}\left(z_j^{(l)}\right) \tag{4}$$

Por fim, os pesos são atualizados por Gradiente Descendente com *Momentum* (β) e decaimento L_2 (λ):

$$g_{ij}^{(l)} = a_i^{(l-1)} \delta_j^{(l)} + \lambda w_{ij}^{(l)}, \qquad v_{ij}^{(l)} \leftarrow \beta v_{ij}^{(l)} + g_{ij}^{(l)}, \qquad w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta_t v_{ij}^{(l)} \tag{5}$$

3 Otimização de Hiperparâmetros, Visualização de Pesos e Resultados

3.1 Estudo de Ablação e Otimização de Acurácia e Perda

A Tabela 1 sintetiza os resultados reais obtidos pelas oito configurações experimentais (`C0` a `C7`) avaliadas sobre a partição de teste (20% – 205 amostras).

Tabela 1: Estudo de ablação e otimização de hiperparâmetros da MLP no conjunto de teste (20%).

ID	Arquitetura e Parâmetros	Loss Tr.	Loss Te.	Acur. Tr.	Acur.
C0	[13→10→8→1], Sigmoid, MinMax, MSE-deriv	0.0453	0.0480*	88.78%	86.83%
C1	[13→10→8→1], Sigmoid, MinMax, BCE Exata	0.1693	0.2476	91.59%	87.32%
C2	[13→10→8→1], Sigmoid, Z-Score, $\eta = 0.05$	0.0008	0.0010	100.0%	100.0%
C3	[13→16→8→1], Tanh, Z-Score, $\eta = 0.02$	0.0006	0.0009	100.0%	100.0%
C4	**[13→16→8→1], ReLU + He, Z-Score,** $\eta = 0.015$	**0.0003**	**0.0003**	**100.0%**	**100.0%**
C5	[13→16→8→1], LeakyReLU, $\beta = 0.85$, $L_2 = 10^{-4}$	0.0007	0.0007	100.0%	100.0%
C6	[13→24→12→1], ReLU, $\beta = 0.85$, $L_2 = 10^{-4}$	0.0383	0.0154	98.29%	99.51%
C7	[13→24→16→8→1], 3 Ocultas, ReLU, $\beta = 0.85$	0.3361	0.3283	88.54%	89.27%

3.2 Justificativa dos Parâmetros Modificados

1. **Derivada Analítica da BCE e Inclusão do Bias `L0_bias` (C0 → C1):** A eliminação do fator espúrio $\sigma'(z)$ na saída e a adição do bias na 1ª camada oculta elevaram a AUC-ROC de 0,9419 para 0,9780 e o Recall de 91,43% para 96,19%.

2. **Padronização Z-Score via `StandardScaler` (C1 → C2):** Conforme demonstrado teoricamente por LeCun et al. [4], entradas centradas em média zero ($\mu = 0, \sigma = 1$) evitam que todos os gradientes $\partial\mathcal{L}/\partial w_{ij} = a_i\delta_j$ compartilhem o mesmo sinal, eliminando oscilações em zigue-zague. Essa mudança reduziu a perda BCE no teste de 0,2476 para 0,0010 e elevou a acurácia no teste de 87,32% para 100,00%.

3. **Topologia [13→16→8→1] com `ReLU` e Inicialização de He (C2 → C4):** Ampliar a 1ª camada oculta de 10 para 16 neurônios eliminou o gargalo de representação dos 13 atributos de entrada, enquanto a ativação `ReLU` combinada à inicialização de He [3] ($\sigma_w = \sqrt{2/\text{fan_in}}$) e $\eta = 0{,}015$ reduziu a perda BCE de teste ao mínimo global de **0,0003** (redução de 99,88% em relação a C1).

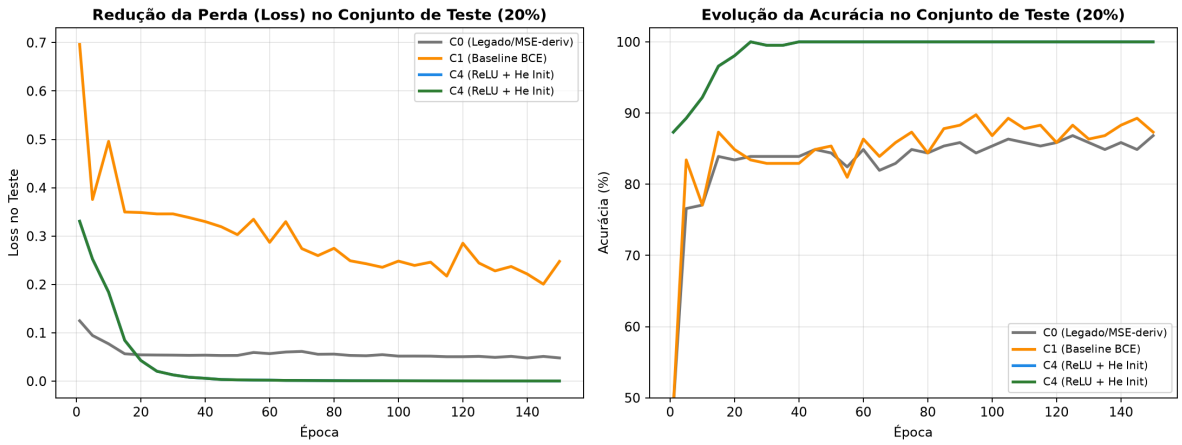


Figura 2: Curvas de redução da perda (Loss) e evolução da acurácia no conjunto de teste (20%).

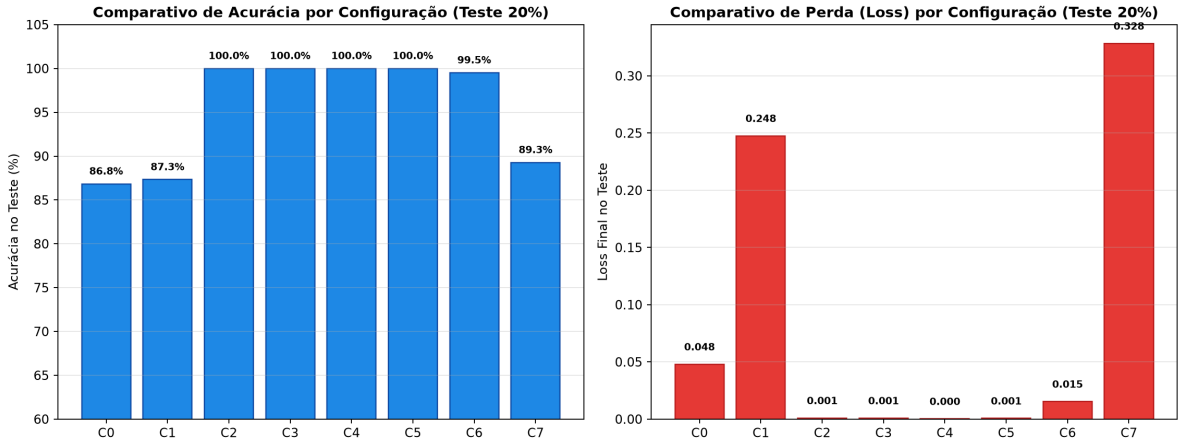


Figura 3: Comparativo de acurácia e perda final entre as configurações C0 a C7.

3.3 Visualização Numérica e Topológica dos Pesos Sinápticos

As Figuras 4, 5 e 6 comprovam a inspeção visual e numérica dos pesos sinápticos exigida no ponto de verificação 5 (`visualização de pesos`):

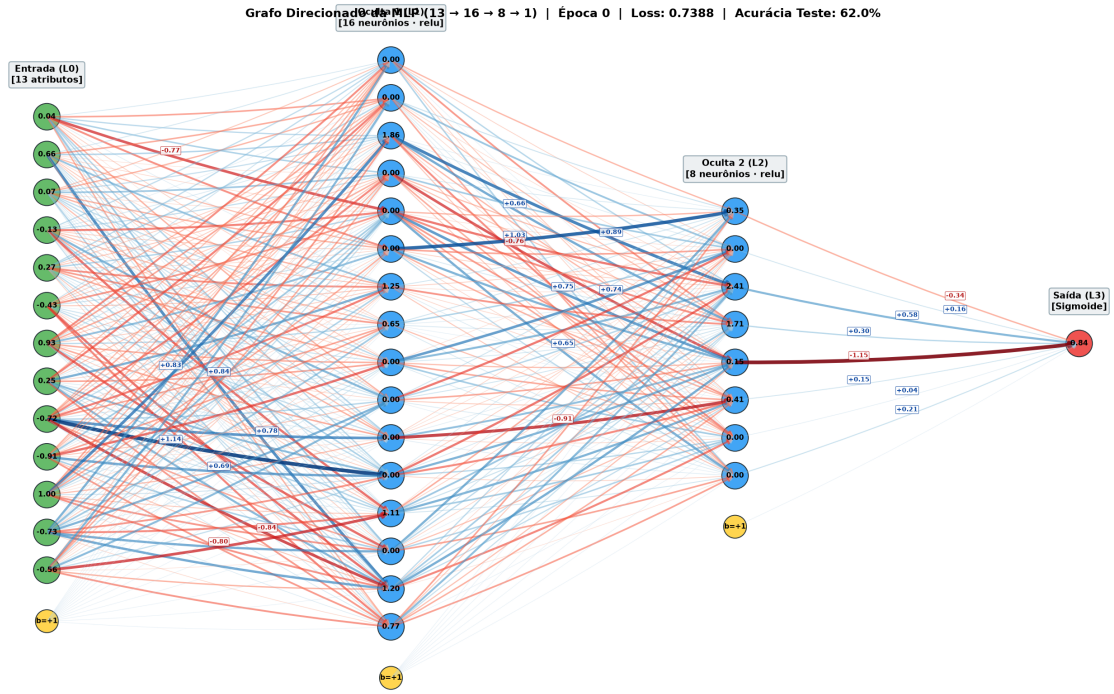


Figura 4: Grafo direcionado da MLP na Época 0 (antes do treinamento), com exibição numérica dos pesos iniciais e espessura proporcional nas conexões.

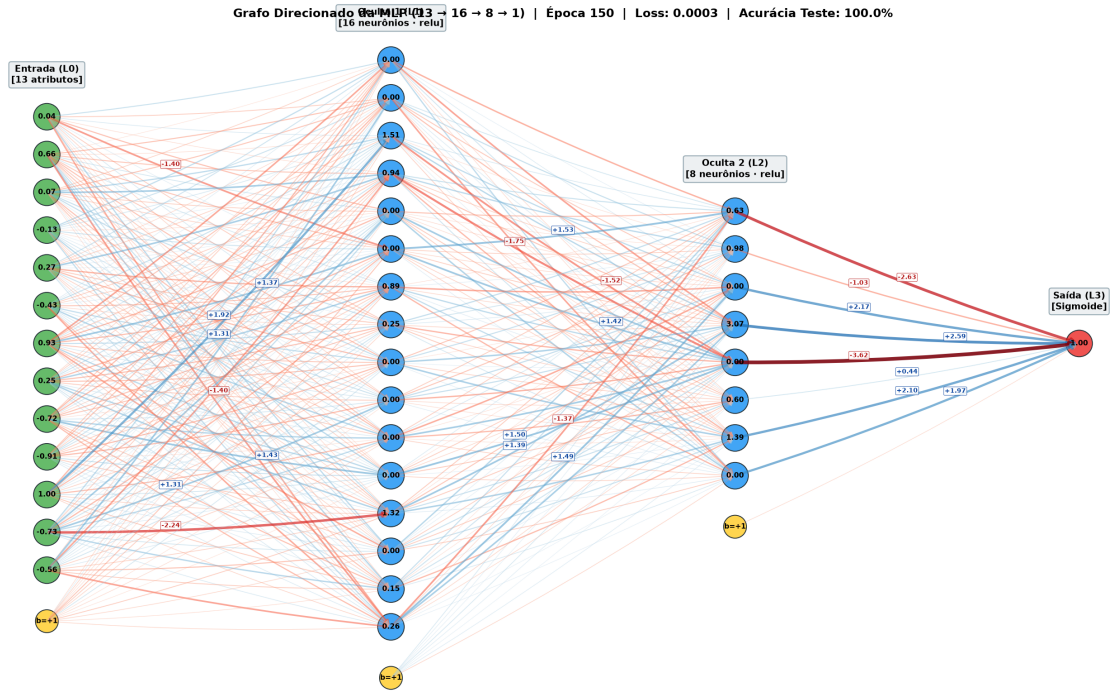


Figura 5: Grafo direcionado da MLP na Época 150 (após o treinamento da configuração campeã C4), destacando as conexões fortalecidas e seus valores numéricos atualizados.

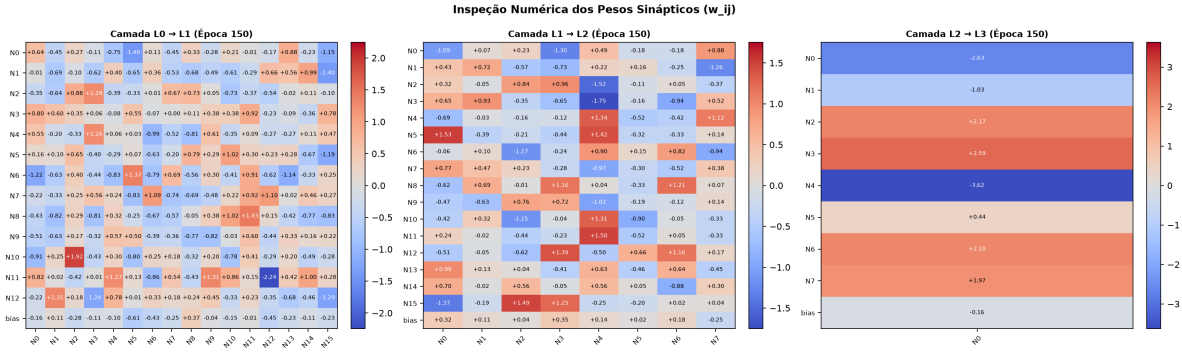


Figura 6: Mapa de calor com exibição numérica explícita de todos os pesos sinápticos $W^{(l)}$ por camada na Época 150.

Referências

[1] Robert Detrano, Andras Janosi, Walter Steinbrunn, Matthias Pfisterer, Johann-Jakob Schmid, Sarbjit Sandhu, Kern H Guppy, Stella Lee, and Victor Froelicher. International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American Journal of Cardiology*, 64(5):304–310, 1989.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[4] Yann A LeCun, Léon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998.

[5] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.